

4.6. Informática

El subsistema informático se compone de los programas y protocolos necesarios para la operación y control de HEXABOT.

El agente principal es el programa “PanelControl1”, desarrollado en lenguaje Java e implementado en la Raspberry PI. En general es responsable de:

- Ofrecer y atender la interfase gráfica al usuario
- Dirigir la secuencia de caminar, incluyendo los cálculos cinemáticos y de trayectoria

Las tres placas Arduino implementan el mismo programa: “ControlArduino3”. Éste se comunica por puerto serie/USB con PanelControl1. En general es responsable de:

- Responder a los comandos provenientes desde PanelControl1
- Llevar la cuenta de la posición de cada actuador
- Encender / apagar los motores para llevarlos hasta el *setpoint*

4.6.1. Protocolo RPI-Arduino (PanelControl1-ControlArduino3)

Entre PanelControl1 y cada ControlArduino3 existe un diálogo basado en un protocolo cliente /servidor, o master/slave. Todos los comandos son originados por PanelControl1 y siempre respondidos por ControlArduino3.

Las características del protocolo son:

- Maestro-esclavo
- ASCII
- Un comando por línea, una respuesta por línea
- Líneas terminadas en “\n” (“new line” o “line feed”; ASCII 10d)
- Cada comando comienza con una letra mayúscula (“A”, “C”, etc.)
- El comando puede completarse con ninguno, uno o dos parámetros numéricos
- Los parámetros numéricos se expresan en base 10, siempre enteros, positivos y separados por espacios
- Las respuestas comienzan con “OK” si el comando fue correctamente interpretado, o “ERR” en caso contrario
- La respuesta se completa con la letra del comando correspondiente
- La respuesta puede incluir uno o más parámetros numéricos, separados por espacios

Comando A, de identificación de placa Arduino

Envío: "A\n"

Respuesta: "OK A " <nro-arduino> "\n"

El número de arduino está grabado internamente en cada placa.

Comando C, de configuración de pines

Envío: "C" <param> <valor> "\n"

Respuesta: "OK C " <param> "\n"

Comando S, de establecimiento de posición de motores:

Envío: "S" <nro-motor> <posicion> "\n"

Respuesta:

"OK S\n" o "ERR\n"

La respuesta del comando es inmediata (asincrónica); no se espera que los motores alcancen la posición indicada antes de la respuesta.

Comando Q, de consulta de posición de motores:

Envío: "Q\n"

Respuesta: "OK Q " <m1> <m2> <m3> <m4> <m5> <m6> <i1> <i2>
<i3> <i4> <i5> <i6> <e1> <e2> <e3> <e4> <e5> <e6>" "\n"

<m1..6> es la cuenta actual del encoder 1 a 6

<i1..6> es el estado actual del sensor de inicio de carrera 1 a 6. Pueden ser "0" o "1".

<e1..6> es el estado actual del sensor del encoder 1 a 6. Pueden ser "0" o "1".

Comando I, de inicio de conteo de encoders:

Envío: "I" <nro-motor> "\n"

Respuesta:

"OK I\n" o "ERR I\n"

La respuesta es inmediata; no se espera que los motores alcancen la posición inicial antes de la respuesta

Comando P, de activación PID

Envío: "P\n"

Respuesta: "OK P\n"

El comando enciende los seis lazos de control. Los actuadores deberán estar completamente configurados (con el comando "C") e inicializados (con el comando "I")

Comando H, de detención inmediata:

Envío: "H\n"

Respuesta:

"OK H\n"

<posicion> es el valor numérico de la posición deseada del actuador (1 a 6) de cada Arduino, en el intervalo [0, 2015] (cálculo aproximado del máximo número de pulsos durante la excursión total del actuador, considerando 24 hilos / pulgada para rosca WW 3/16"; aproximadamente 6 pulgadas es el largo de la varilla roscada; el encoder genera 14 pulsos / vuelta).

<nro-motor> es el número de motor, entre 1 y 6.

<nro-arduino> es el número de placa Arduino: 1=control de coxas; 2=control de patas 1 a 3; 3=control de patas 4 a 6.

<param1> es el número de parámetro

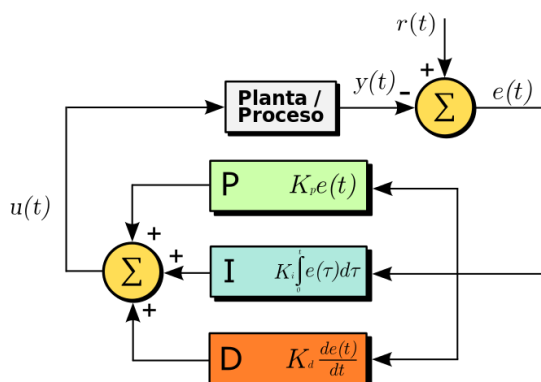
1=MOT-1+; 2=MOT-2+; 3=MOT-3+; 4=MOT-4+; 5=MOT-5+; 6=MOT-6+;
11=MOT-1-; 12=MOT-2-; 13=MOT-3-; 14=MOT-4-; 15=MOT-5-; 16=MOT-6-;
21=ENC-1; 22=ENC-2; 23=ENC-3; 24=ENC-4; 25=ENC-5; 26=ENC-6;
31=FIN-1; 32=FIN-2; 33=FIN-3; 34=FIN-4; 35=FIN-5; 36=FIN-6;
41=LED-R-1; 42=LED-R-2; 43=LED-R-3;
51=LED-G-1; 52=LED-G-2; 53=LED-G-3;
61=LED-B-1; 62=LED-B-2; 63=LED-B-3;
71=PIS-1; 72=PIS-2; 73=PIS-3;
81=MAX-1; 82=MAX-2; 83=MAX-3; 84=MAX-4; 85=MAX-5; 86=MAX-6;
91=KP-1; 92=KP-2; 93=KP-3; 94=KP-4; 95=KP-5; 96=KP-6;
101=KI-1; 102=KI-2; 103=KI-3; 104=KI-4; 105=KI-5; 106=KI-6;
111=KD-1; 112=KD-2; 113=KD-3; 114=KD-4; 115=KD-5; 116=KD-6;

<valor> Número de pin, según nomenclatura Arduino, como la utilizada en la función “setMode”; o valor máximo de excursión; o factor P/I/D (en milésimas)

4.6.2. Sistema de control PID

Un controlador PID es un mecanismo de control por realimentación ampliamente usado en sistemas de control industrial. Este calcula la desviación o error entre un valor medido y un valor deseado.

El algoritmo del control PID consiste de tres parámetros distintos: el proporcional, el integral, y el derivativo. El valor Proporcional depende del error actual. El Integral depende de los errores pasados y el Derivativo es una predicción de los errores futuros. La suma de estas tres acciones es usada para ajustar al proceso por medio de un elemento de control como la posición de una válvula de control o la potencia suministrada a un calentador. [1]

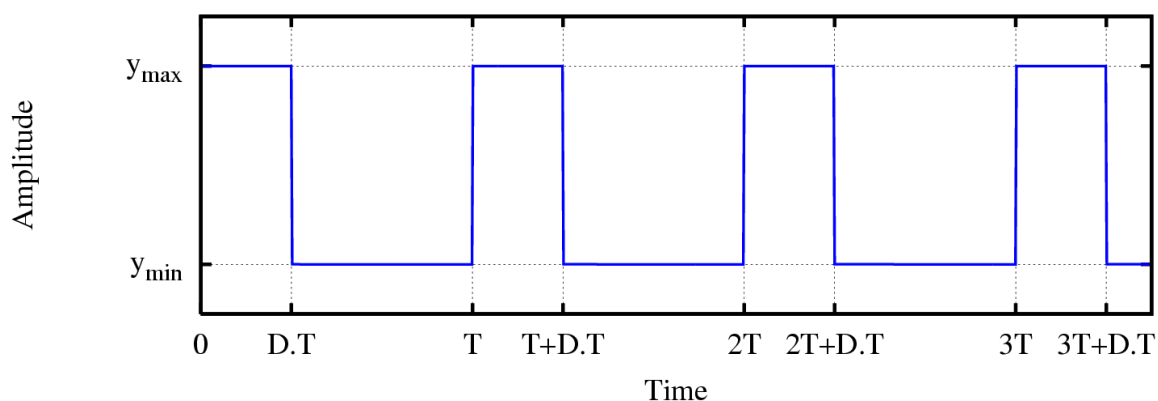


El programa ControlArduino3 utiliza el algoritmo PID de doble efecto, para establecer la posición: la variable a controlar es la cantidad de pulsos del encoder (setpoint). En las coxas, puede variar entre 0 y 4000. En los fémures y tibias, entre 0 y 1500.

La variable sobre la que se actúa es la tensión aplicada sobre el motor, positiva o negativa.

4.6.3. PWM

La modulación por ancho de pulsos (también conocida como PWM, siglas en inglés de *pulse-width modulation*) de una señal o fuente de energía es una técnica en la que se modifica el ciclo de trabajo de una señal periódica (una senoidal o una cuadrada, por ejemplo), ya sea para transmitir información a través de un canal de comunicaciones o para controlar la cantidad de energía que se envía a una carga. [2]



ControlArduino3 utiliza dos líneas PWM para cada motor, lográndose así una tensión pseudo-analógica positiva o negativa, de 0 a 255 pulsos (-24 a 24V nominales).

4.6.4. Algoritmos de movimiento

A diferencia de un robot móvil con ruedas, la locomoción basada en patas exige a la unidad de control la dirección simultánea de numerosos motores (en HEXABOT, 18 simultáneos), cálculos de trayectoria, conversiones cartesianas / polares y finalmente traducirlo a valores de los actuadores lineales.

Sentado (*sit*)

En esta maniobra, cada actuador se contrae hasta alcanzar el inicio de carrera. Sirve para iniciar en cero la cuenta de pulsos de los encoders. Se ejecuta a máxima potencia:

1. primero suben los seis fémures simultáneos
2. luego se contraen las tibias
3. finalmente rotan en sentido horario las coxas

PanelControl1 envía los siguientes comandos de inicialización, que son ejecutados en ControlArduino3:

Arduino	Comandos	Actuador
2	I 1 I 3 I 5	Fémur 1 Fémur 2 Fémur 3
3	I 1 I 3 I 5	Fémur 4 Fémur 5 Fémur 6
Pausa 1500mS		
2	I 2 I 4 I 6	Tibia 1 Tibia 2 Tibia 3
3	I 2 I 4 I 6	Tibia 4 Tibia 5 Tibia 6
Pausa 1500mS		
1	I 1 I 2 I 3 I 4 I 5 I 6	Coxa 1 Coxa 2 Coxa 3 Coxa 4 Coxa 5 Coxa 6

Parado (*up*)

En esta maniobra, se apoyan en el piso las seis patas, en los puntos centrales de pisada. Cada coxa se mueve hasta el ángulo relativo 0° (centro de la excursión, coincidente con el radio característico de la pata). Las tibias caen verticalmente desde la rodilla hasta el piso. Los fémures se inclinan lo necesario hasta lograr la elevación del cuerpo solicitada.

Sólo cuando la elevación solicitada es 410mm, los fémures se posicionan horizontalmente.

Durante la maniobra “up”, PanelControl1 establece para cada pata el “punto central de pisada”.

Caminar (*walk*)

Para caminar, PanelControl1 proyecta una sucesión de pasos. Cada paso se descompone en dos semipasos: semipaso de avance y semipaso de retroceso.

Se requiere conocer el rumbo, la velocidad, la longitud del paso y la altura de despeje.

Durante el semipaso de avance, tres patas (por ejemplo, 1-3-5) describen una trayectoria elíptica desde el punto trasero del paso, hacia el punto delantero del paso.

Simultáneamente al semipaso de avance, las otras tres patas (2-4-6) ejecutan el semipaso de retroceso, describiendo una trayectoria recta, en contacto con el piso, desde el punto delantero del paso, hacia el trasero.

Al concluir estos semipasos, comienzan nuevamente intercambiándose las patas: ahora 1-3-5 realizan el semipaso de retroceso, y 2-4-6 el de avance.

La secuencia continúa hasta que la velocidad llega a cero, o se ordena “Detener”.

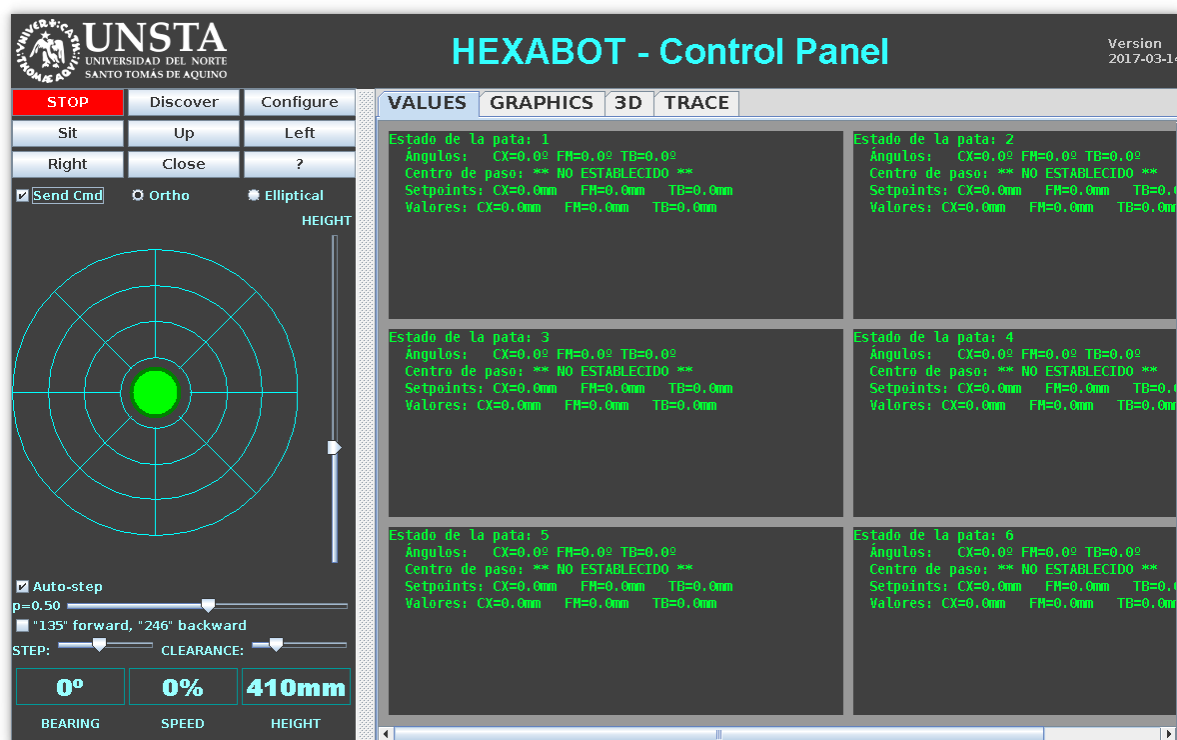
Detener (*stop*)

Para detener los actuadores, se envía simultáneamente el comando “H” (halt) a cada Arduino. Se apagan los motores y desactivan los lazos PID.

4.6.5. Interfase de usuario

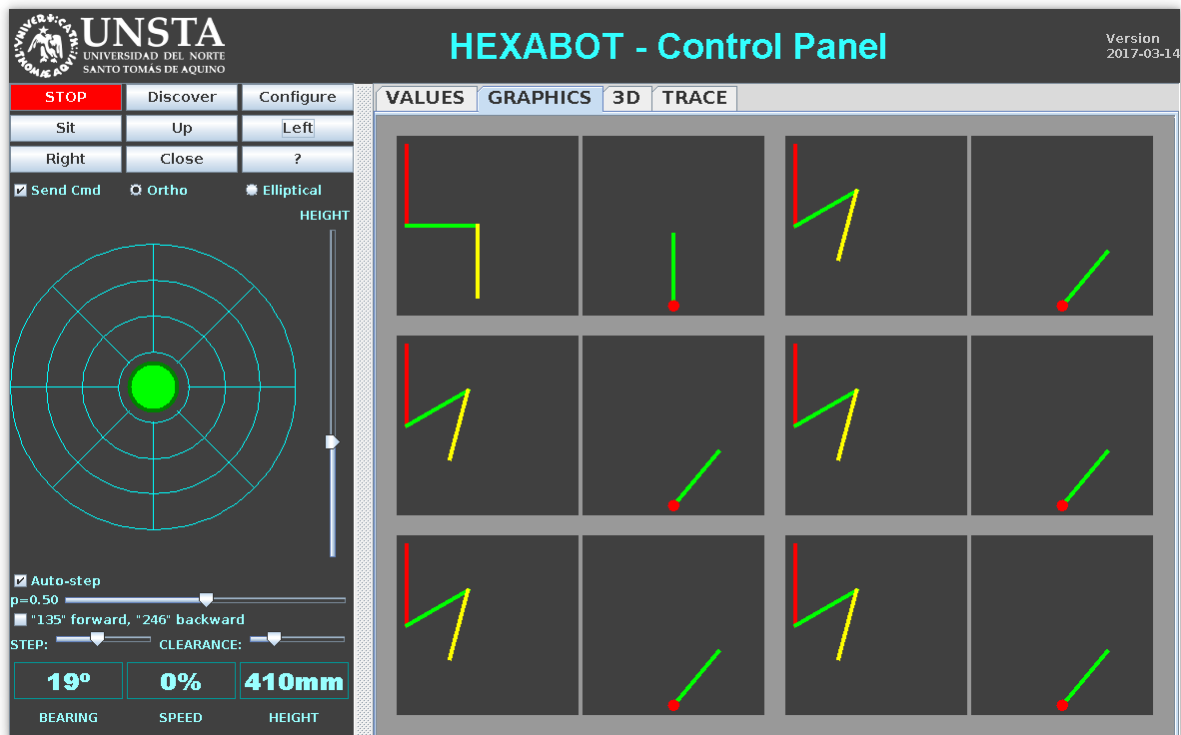
La interfase de usuario desplegada por PanelControl1 utiliza la librería Swing, además de dos componentes específicamente desarrollados:

- El joystick para control de rumbo y velocidad
- El visualizador tridimensional del modelo

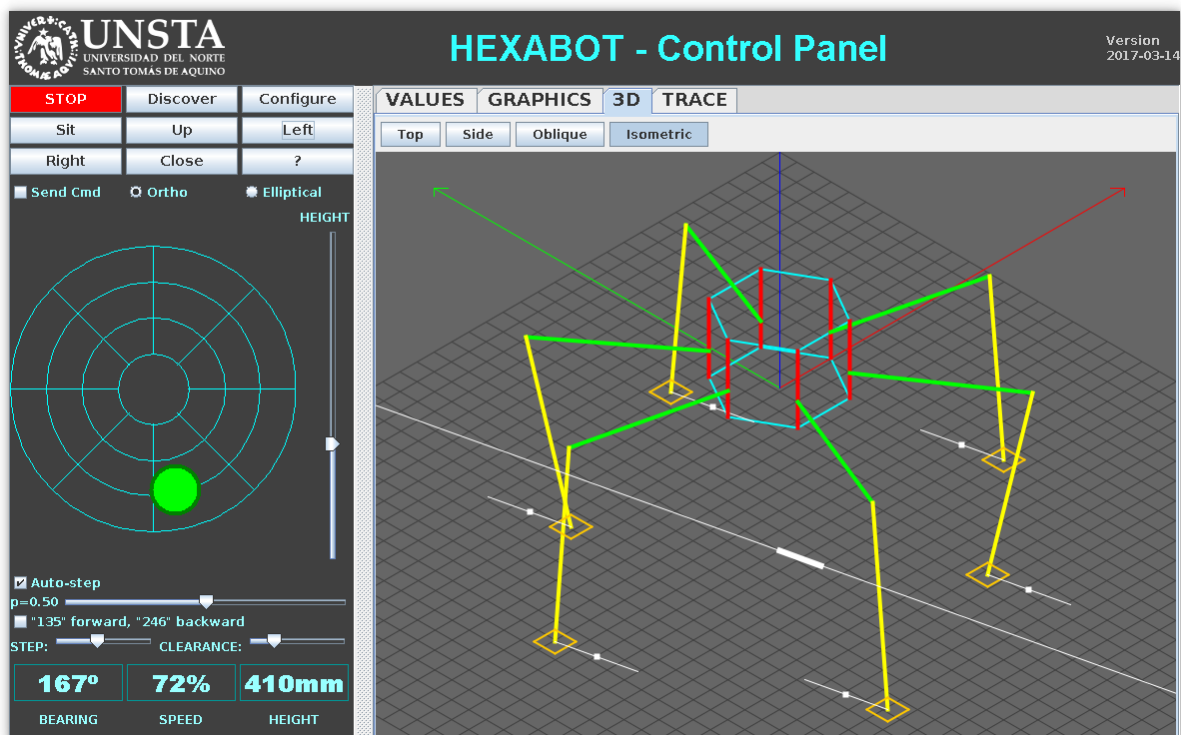


La única pantalla de PanelControl1 se divide en dos: a la izquierda, todos los comandos. A la derecha, cuatro pestañas que muestran el estado del robot.

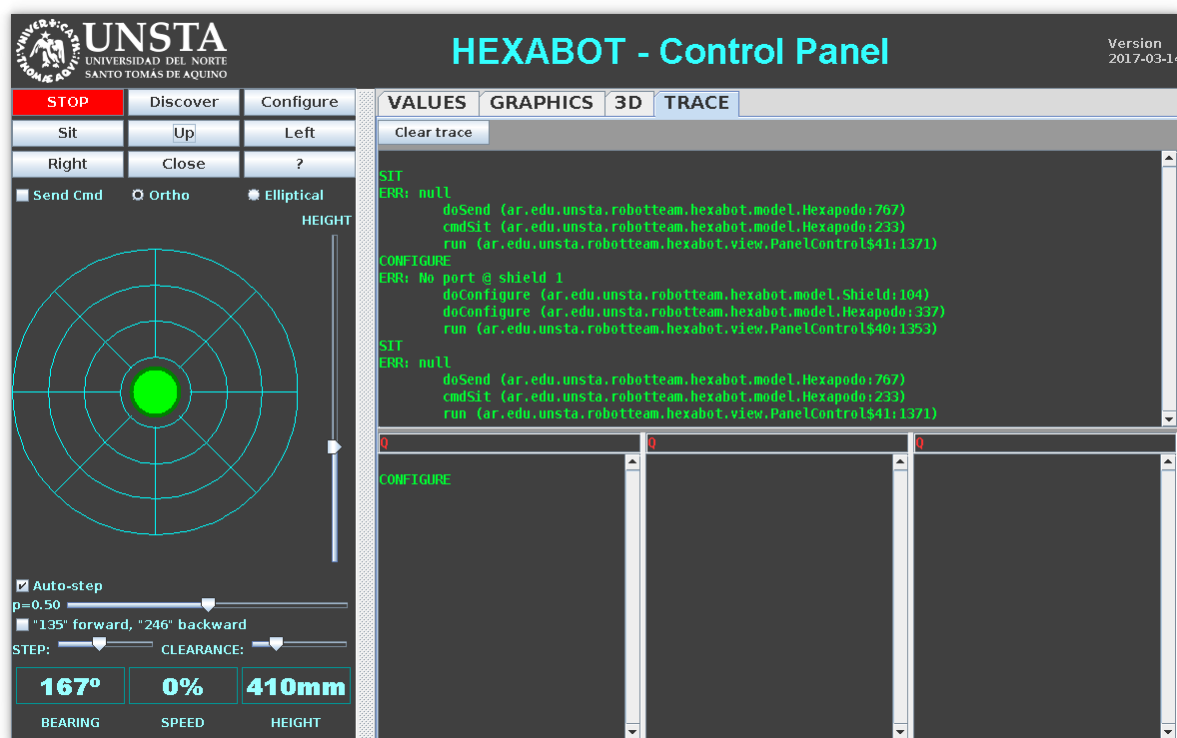
La primera pestaña “VALUES” muestra los valores deseados (no reales) de cada extremo de pata.



La pestaña “GRAPHICS” muestra una representación esquemática del estado de los tres actuadores de cada pata.



La pestaña “3D” ofrece una representación en malla de alambre del modelo. Los cuadrados amarillos indican cuando la pata está en contacto con el piso. Los segmentos blancos en cada pata indican el punto central de pisada, el punto delantero del paso y el punto trasero del paso.



La pestaña “TRACE” lleva un registro general del programa, y registros individuales por cada Arduino, de los comandos enviados y recibidos.

Esta pestaña también permite enviar comandos libres a cada Arduino. El más común es “Q” para consultar el estado actual de los actuadores.

4.6.6. Código fuente de PanelControl1

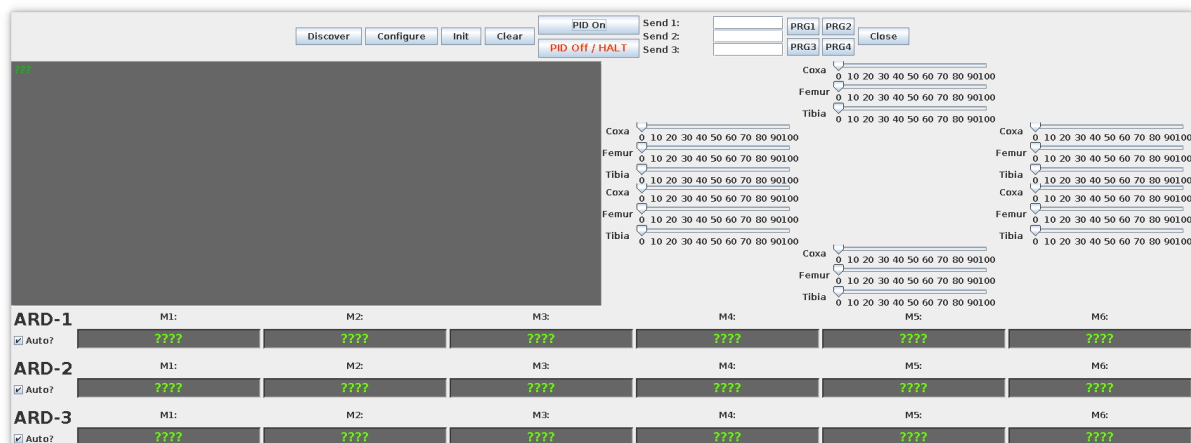
Todo el código fuente implementa clases del paquete “ar.edu.unsta.robotteam.hexabot”.

El subpaquete “model” contiene las clases que modelan al robot. La principal, de instancia única es “Hexapodo”. Tiene seis miembros de clase “Pata” y tres de clase “Shield”.

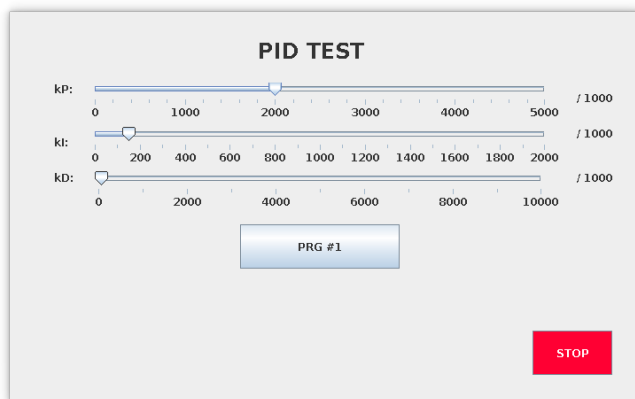
Cada pata tiene seis miembros “CanalShield”.

Cada pata tiene dos miembros de clase “Actuador”. Cada actuador se asocia a un “CanalShield”.

“test” incluye programas de ensayos no requeridos en la producción final



El programa “CommTest” se utilizó para ensayos a bajo nivel.



El programa “PIDTest” permitió ensayar los parámetros P/I/D.

“util” incluye clases de servicio para la interacción con los puertos serie, y librerías matemáticas

“view” incluye los artefactos de la interfase de usuario

4.6.7. Código fuente de ControlArduino3

El archivo “ControlArduino3.ino” contiene el código fuente del programa a bordo de las tres placas. Sólo difieren en el número de identificación de cada una (“1” para coxas, “2” para patas 1 a 3 y “3” para patas 4 a 6).

Se programó utilizando la técnica de “multitarea cooperativa”, para dar actividad simultánea a las siguientes tareas concurrentes:

- Parser de comandos recibidos por el puerto serie
- Conteo de pulsos del encoder
- Ejecución de la tarea INIT
- Control PID/PWM

Referencias

- [1] https://es.wikipedia.org/wiki/Controlador_PID
- [2] https://es.wikipedia.org/wiki/Modulaci%C3%B3n_por_ancho_de_pulsos